

DART PROGRAMMING

A Complete Beginner's Guide

Syntax • Operators • Data Types • Variables • OOP

Chapter 1: What Is Dart?

Dart is a programming language created by Google. Think of it like a set of instructions you can write to tell a computer what to do. Just like English has grammar rules, Dart has its own rules called syntax. It is used to build mobile apps (especially with Flutter), web apps, and server programs.

Dart looks a lot like other popular languages such as Java and JavaScript, so if you have seen those before, Dart will feel familiar.

1.1 Dart vs JavaScript — What's the Difference?

Here's a simple comparison between Dart and JavaScript:

Feature	Dart	JavaScript
Type System	Optional – you can choose to declare types or not	Weak and dynamic – types are loosely enforced
Classes	Yes – supports single inheritance (one parent class)	Prototype-based (older style)
Interfaces	Yes – supports multiple interfaces	No built-in interface support
Concurrency	Yes – uses isolates	Yes – uses web workers

1.2 Running Dart Online (No Setup Needed)

You can try Dart code immediately without installing anything by visiting:

<https://dartpad.dev/>

Type your code there and click Run. This is perfect for beginners who just want to experiment.

Chapter 2: Dart Syntax — The Rules of Writing Dart

Syntax is like the grammar of a programming language. Just as English has rules (sentences end with a period, questions end with a question mark), Dart has rules too. If you break these rules, the computer will not understand your code and will show an error.

2.1 Your First Dart Program

Every Dart program starts with a special function called `main()`. A function is just a block of code that performs a task. The `main()` function is where your program starts running.

```
main() {  
    print("Hello World!");  
}
```

Let's break this down word by word:

- **`main()`** — This is the entry point. Dart always starts here.
- **`{ }`** — The curly braces wrap the code that belongs to `main()`.
- **`print(...)`** — This is a built-in command that prints text to the screen.
- **`"Hello World!"`** — This is the text we want to show. Text must be in quotes.
- **`;`** — Every statement (instruction) in Dart ends with a semicolon.

□ **Note:** Output of the program above: Hello World!

2.2 How to Run a Dart Program

There are two ways to run Dart code:

Option 1: Using the Terminal (Command Line)

If you have Dart installed on your computer, open the terminal and type:

```
dart file_name.dart
```

Replace `file_name.dart` with the actual name of your file.

Option 2: Using WebStorm IDE

An IDE (Integrated Development Environment) is a special program that makes coding easier. WebStorm is one such tool. To run your code in WebStorm:

- Right-click on your Dart file
- Click 'Run <file_name>'
- Or press `Ctrl + Shift + F10` on your keyboard

2.3 Dart Command-Line Options

When running Dart from the terminal, you can add special switches called command-line options. These change how your program runs:

Option	What It Does
-c or --checked	Turns on 'checked mode' — Dart will warn you about type errors
--version	Shows which version of Dart you have installed
--packages <path>	Tells Dart where to find package configuration files
-p <path>	Tells Dart where to find imported libraries
-h or --help	Shows help information about available options

2.4 Checked Mode vs Production Mode

Dart can run in two different modes:

- **Checked Mode** — Dart is strict. It checks for type errors and warns you. Great for development and learning.
- **Production Mode** — Dart is fast. It skips extra checks for better performance. Use this when your app is ready.

Example: This code assigns a text value to a number variable (which is wrong):

```
void main() {
  int n = "hello";    // 'int' means number, but we gave it text!
  print(n);
}
```

In production mode (default), this will actually run and print 'hello'. But in checked mode:

```
dart -c Test.dart
```

Dart will stop and show an error like:

```
type 'String' is not a subtype of type 'int'
```

□ **Note:** Always use checked mode while learning. It will catch your mistakes early!

2.5 Identifiers — Naming Things in Dart

An identifier is a name you give to something in your program, like a variable (a container for data), a function, or a class. Think of it like naming a folder on your computer.

Rules for Naming Identifiers

- Can contain letters (A-Z, a-z) and numbers (0-9)
- Cannot start with a number
- Cannot contain spaces
- Cannot use special characters except underscore `_` and dollar sign `$`
- Cannot use reserved keywords (words Dart already uses, like 'if', 'class', 'var')
- Are case-sensitive: `firstName` and `firstname` are different!

Examples of valid and invalid identifiers:

Valid (Allowed)	Invalid (Not Allowed)
firstName	Var (this is a keyword)
first_name	first name (has a space)
num1	first-name (has a hyphen)
\$result	1number (starts with a number)

2.6 Keywords — Words Reserved by Dart

Keywords are special words that Dart already uses for specific purposes. You cannot use them as variable names. Some common keywords include:

Keywords					
abstract	class	false	if	return	true
assert	const	final	import	static	var
break	continue	for	new	super	void
case	default	null	switch	while	try

2.7 Other Important Syntax Rules

Dart Ignores Extra White Space

Dart does not care about extra spaces, tabs, or blank lines. This is fine:

```
main() { print("Hi"); }
```

And this is also fine (same program, just formatted nicely):

```
main() {  
    print("Hi");  
}
```

Dart Is Case-Sensitive

Dart treats uppercase and lowercase letters as completely different. For example, Print and print are two different things. Only print (lowercase) is a built-in function.

Statements End with a Semicolon

Every single instruction (called a statement) must end with a semicolon ;. Forgetting it is one of the most common beginner mistakes!

```
print("Hello"); // Correct  
print("Hello") // Wrong! Missing semicolon
```

2.8 Comments — Notes for Humans

Comments are notes you write in your code to explain what it does. Dart ignores comments completely — they are only for humans reading the code.

Single-Line Comment

Use `//` to comment out a single line:

```
// This is a note. Dart ignores this line.  
print("Hello"); // This prints Hello
```

Multi-Line Comment

Use `/* ... */` to comment out multiple lines at once:

```
/*  
    This is a longer comment.  
    It can span multiple lines.  
*/
```

Chapter 3: Data Types — What Kind of Data Can Dart Store?

Every piece of data has a type. Dart needs to know what kind of data you're working with so it can handle it properly. Think of data types like different kinds of containers — a jar for liquid, a box for solids, etc.

3.1 Numbers

- **int** — Stores whole numbers (no decimal). Example: 10, -5, 100
- **double** — Stores numbers with decimal points. Example: 3.14, 9.99, -0.5

```
int age = 25;  
double price = 9.99;
```

3.2 Strings

A String is text — a sequence of characters. You use it to store names, addresses, messages, anything that is words.

```
String name = 'Ahmed';  
String city = "Karachi";
```

□ **Note:** You can use single quotes `' '` or double quotes `" "` — both work!

3.3 Booleans

A Boolean can only be one of two things: true or false. It's like a light switch — on or off.

```
bool isLoggedIn = true;
bool hasError = false;
```

3.4 Lists

A List is an ordered collection of items — like a shopping list. All items are in order and you can access them by their position (called index, starting from 0).

```
List<String> fruits = ['apple', 'banana', 'mango'];
print(fruits[0]); // Output: apple
```

3.5 Maps

A Map stores data in key-value pairs — like a dictionary where each word (key) has a definition (value).

```
Map<String, int> scores = {'Ali': 90, 'Sara': 85};
print(scores['Ali']); // Output: 90
```

3.6 Dynamic Type

If you don't specify a type, Dart uses 'dynamic' — the variable can hold any type of data.

```
dynamic anything = 42;
anything = "now it's text"; // This is allowed!
```

Chapter 4: Variables — Storing Data in Dart

A variable is like a labeled box. You put data in it, and you can use the label to find and use that data later.

4.1 Declaring Variables

Use the var keyword to declare (create) a variable:

```
var name = 'Smith';
```

Or you can specify the type explicitly (more precise, recommended for beginners):

```
String name = 'Smith';
int age = 30;
double salary = 50000.50;
```

4.2 Uninitialized Variables Are null

If you declare a variable but don't give it a value, it holds a special value called null (meaning 'nothing' or 'empty').

```
void main() {
  int num;
```

```
print(num); // Output: null
}
```

4.3 final and const — Variables That Can't Change

Sometimes you want a value that never changes — like the value of pi (3.14159...). Use final or const:

final

A final variable is set once and cannot be changed after that. Its value is determined when the program runs.

```
final city = 'Karachi';
city = 'Lahore'; // ERROR! Cannot change a final variable
```

const

A const variable must be known at compile time (before the program even runs). It is also permanent.

```
const pi = 3.14;
const area = pi * 12 * 12; // Valid: computed from other constants
print(area); // Output: 452.16
```

□ **Note:** Use const when you know the value at the time of writing code. Use final when the value is set at runtime but won't change.

Keyword	Can Change?	When is value set?
var	Yes, anytime	At runtime
final	No (set once)	At runtime
const	No (set once)	At compile time

Chapter 5: Operators — Doing Things With Data

Operators are symbols that perform operations on data. For example, + adds two numbers, == checks if two values are equal. The data being operated on are called operands.

Example: In the expression 2 + 3, the numbers 2 and 3 are operands, and + is the operator.

5.1 Arithmetic Operators — Math Operations

Operator	Name	Example
+	Addition	5 + 3 = 8
-	Subtraction	10 - 4 = 6

Operator	Name	Example
*	Multiplication	3 * 4 = 12
/	Division (decimal)	10 / 3 = 3.333...
~/	Integer Division	10 ~/ 3 = 3 (no decimal)
%	Modulo (Remainder)	10 % 3 = 1 (remainder after dividing)
++	Increment	num++ increases num by 1
--	Decrement	num-- decreases num by 1

5.2 Relational (Comparison) Operators — Comparing Values

These operators compare two values and always return either true or false.

Example: A = 10, B = 20

Operator	Meaning	Example	Result
>	Greater than	A > B	false (10 is not > 20)
<	Less than	A < B	true (10 < 20)
>=	Greater than or equal	A >= B	false
<=	Less than or equal	A <= B	true
==	Equal to	A == B	false (10 is not 20)
!=	Not equal to	A != B	true (10 is different from 20)

5.3 Type Test Operators — Checking the Type

These operators check what data type a variable holds:

Operator	What It Does
is	Returns true if the object is of the given type. Example: x is String
is!	Returns true if the object is NOT of the given type. Example: x is! int

5.4 Assignment Operators — Storing Values

Assignment operators store or update values in variables:

Operator	Example	Meaning
=	c = 5	Assign 5 to c
??=	c ??= 5	Only assign 5 if c is currently null
+=	c += 3	Same as: c = c + 3
-=	c -= 3	Same as: c = c - 3
*=	c *= 3	Same as: c = c * 3
/=	c /= 3	Same as: c = c / 3

5.5 Logical Operators — Combining Conditions

Logical operators combine multiple conditions and return true or false. Example: A = 10, B = 20

Operator	Meaning	Example
&&	AND — both conditions must be true	(A > 5 && B > 5) = true
	OR — at least one condition must be true	(A > 15 B > 15) = true
!	NOT — reverses the condition	!(A > 15) = true (since A is not > 15)

5.6 Conditional Expressions — Shortcuts for if/else

These let you write simple decisions in one line instead of writing a full if-else block.

Ternary Operator: condition ? value_if_true : value_if_false

```
void main() {
    var a = 10;
    var result = a > 12 ? "greater than 12" : "12 or less";
    print(result); // Output: 12 or less
}
```

Think of it as: 'If a > 12, use the first option; otherwise use the second.'

Null-Check Operator: expr1 ?? expr2

```
void main() {
    var a = null;
    var b = 12;
    var result = a ?? b; // a is null, so use b
    print(result); // Output: 12
}
```

```
}
```

This says: 'If the first value is not null, use it. Otherwise, use the second.'

Chapter 6: Decision Making — Telling Dart What to Do

Programs often need to make decisions. Should I show this message? Should I add this item? Decision-making in Dart is done using conditional constructs.

6.1 The if Statement

Run a block of code only if a condition is true.

```
void main() {  
    var num = -5;  
    if (num < 0) {  
        print("number is negative");  
    }  
}
```

Since -5 is indeed less than 0, the condition is true and the message prints.

□ **Note:** Output: number is negative

6.2 The if...else Statement

Do one thing if the condition is true, and something else if it's false.

```
if (num > 0) {  
    print("Positive");  
} else {  
    print("Not positive");  
}
```

6.3 The else if Ladder

Check multiple conditions in order:

```
if (score >= 90) {  
    print("A grade");  
} else if (score >= 70) {  
    print("B grade");  
} else {  
    print("Below B");  
}
```

6.4 The switch Statement

The switch statement is useful when you have many possible values to check for a single variable:

```
var day = "Monday";
switch (day) {
  case "Monday":
    print("Start of the week!");
    break;
  case "Friday":
    print("Almost weekend!");
    break;
  default:
    print("Midweek day");
}
```

□ **Note:** The 'break' keyword is important — it stops Dart from falling into the next case.

Quick Reference Summary

Concept	Key Takeaway
Syntax	Rules for writing Dart code. Statements end with ;
main()	Every Dart program starts here
print()	Displays text in the terminal
Comments	// single line, /* */ multi-line. Dart ignores them.
Identifiers	Names for variables/functions. No spaces, no starting digits.
Data Types	int, double, String, bool, List, Map, dynamic
var	Declare a variable without specifying type
final / const	Variables that cannot be changed after being set
Arithmetic ops	+ - * / ~/ % ++ --
Relational ops	> < >= <= == != (return true/false)
Logical ops	&& (AND) (OR) ! (NOT)
Ternary op	condition ? value_if_true : value_if_false
?? operator	Use second value if first is null
if / else	Make decisions based on conditions

Concept	Key Takeaway
Class	Blueprint for creating objects (OOP)
Object	An instance (copy) of a class

Happy Coding with Dart! ☐